PROJECT DOCUMENTATION FOR FINANCE TRACKER

1 CONTENTS

2 Introduction & Problem Definition 3
2.1 Introduction 3
2.2 Problem Statement 3
2.3 Key Features 3
3 Finance Tracker Basics 4
3.1 Starting Finance Tracker 4
3.2 The parts of Finance Tracker 4
3.3 How to use 5
4 Design and development details 6
4.1 Tech Stack 6
4.2 Framework & Pattern 6
4.3 Core Components 6
4.3.1 Database Schema Room 6
4.3.2 Data Access Objects (DAOs) 7
4.3.3 Core Logic 7
4.4 Status and Enum Definitions 7
5 Implementation Details 7
5.1 Adding a Record 7
5.2 Monitoring & Calculation 7
5.3 Settling a Debt 7

2 INTRODUCTION & PROBLEM DEFINITION

2.1 INTRODUCTION

Finance Tracker is an Android-based financial utility designed to help users track personal debts (both owed and owing). Developed using Java and the Android Jetpack suite, the application provides a centralized interface for recording who owes money to the user and vice versa. By bridging the gap between casual verbal agreements and formal accounting, the app ensures financial transparency and reduces the risk of forgotten obligations.

2.2 PROBLEM STATEMENT

In personal and micro-business environments, tracking "who owes whom" is often managed through memory or disorganized notes. This leads to:

- ✓ Forgotten debts.
- ✓ Inaccurate balance tracking (mixing receivables and payables).
- ✓ Lack of a clear timeline for due dates.

2.3 KEY FEATURES

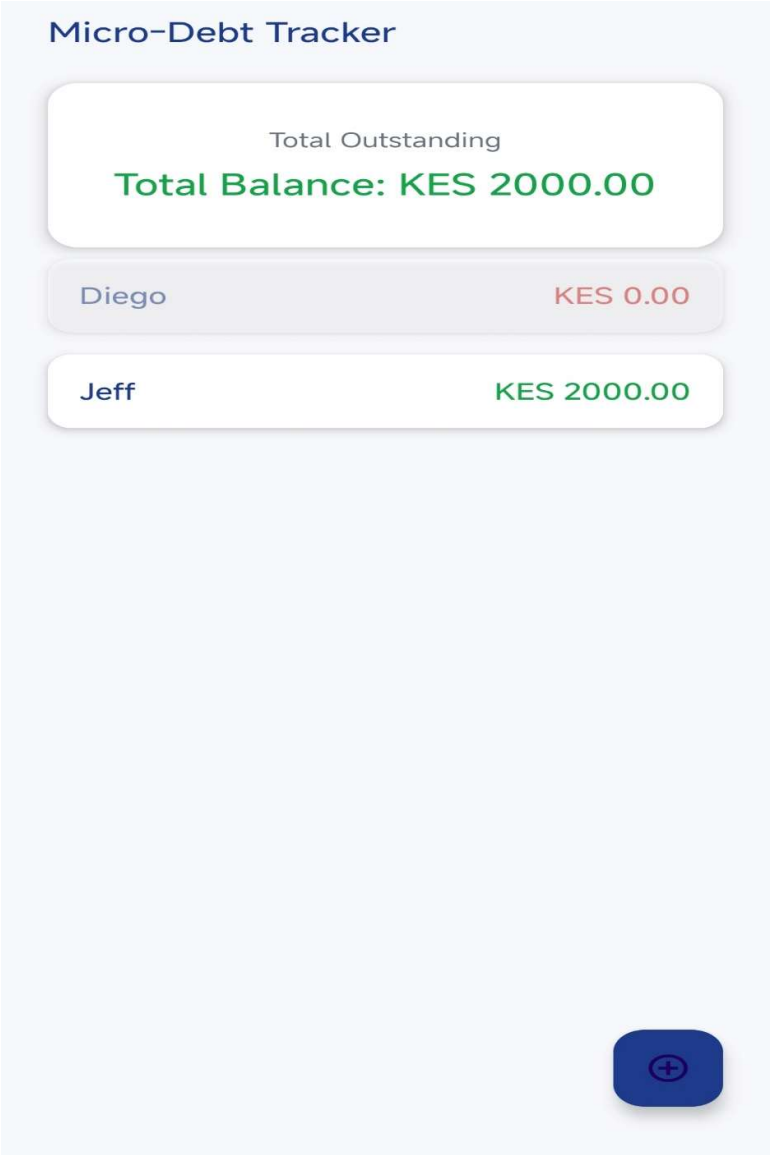
- Dual Debt Tracking: Supports both "Owed to Me" (receivables) and "I Owe" (payables) categories.
- Real-time Balance Calculation: Automatically calculates the net balance (Receivables - Payables) in KES (Kenyan Shillings).
- Settlement Lifecycle: Allows users to mark debts as fully paid or track partial payments, with visual cues for settled items
- Detailed Records: Stores metadata including person names, amounts, due dates (to-do), payment methods (Cash, Mobile Money, Bank Transfer), and descriptions.
- Local Persistence: Uses an on-device database to ensure data is accessible without an internet connection.

3 FINANCE TRACKER BASICS

3.1 STARTING FINANCE TRACKER

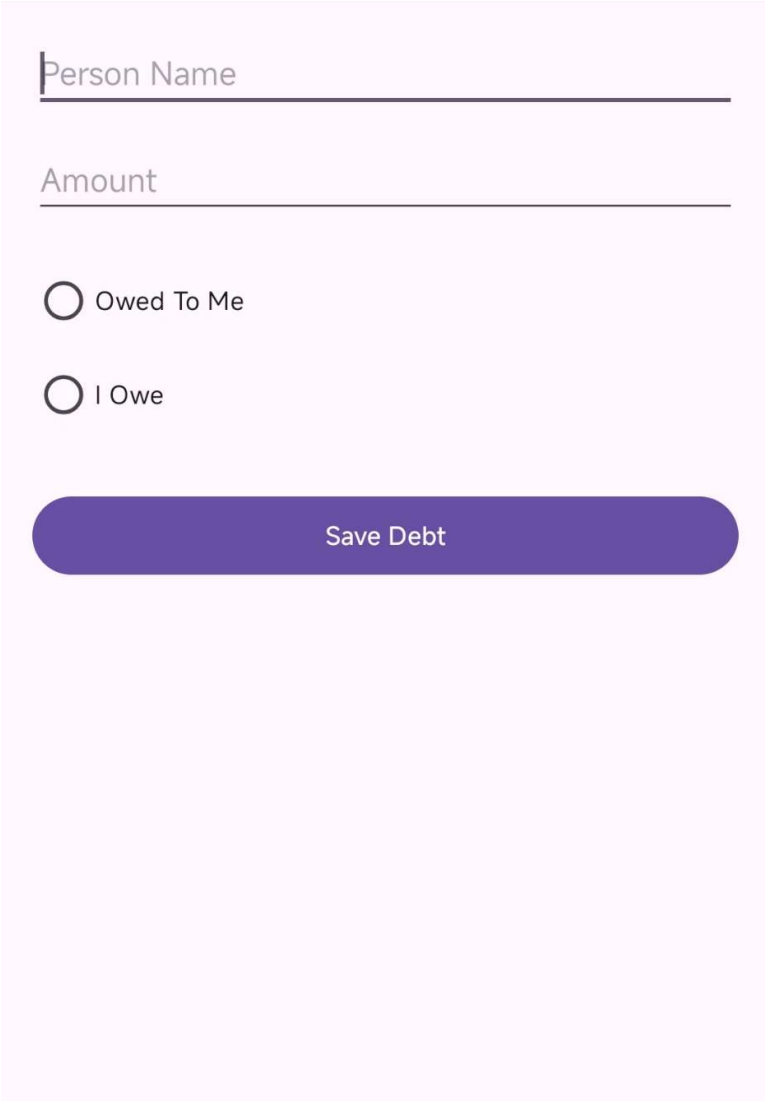
After install, click the financial tracker icon from your phone's list of apps to run it.

3.2 THE PARTS OF FINANCE TRACKER



A brief look into Finance Tracker's main window reveals the following:

- ✓ App alternative name at the top left.
- ✓ A bar/pill that shows the total outstanding cumulative debts i.e., the total of debts owed and dets owing.
- ✓ A plus button that opens another screen, allowing you to add debts.



- ✓ Person Name and Amount field that take the person's name and the amount of debt (in Ksh).
- ✓ Owed to Me/ I Owe radio buttons to specify the nature of the debt.

3.3 HOW TO USE

- To add a new debt, press the plus button, add the corresponding debt details and press save.
- To mark a debt as paid, long press it and click "mark as paid' from the appearing context menu.
- To delete a debt, long press it and click "delete' from the appearing context menu.
- Long press to see a summary of additional details.
- Debts you owe appear in red while debts that are owed to you appear in green.

4 DESIGN AND DEVELOPMENT DETAILS

4.1 TECH STACK

- ✓ Language: Java
- ✓ Database: SQLite via Room Persistence Library (an abstraction layer over SQLite).
- ✓ UI Components: Material Design 3, RecyclerView, Floating Action Buttons (FAB), and ViewBinding/ConstraintLayouts.
- ✓ Architecture Pattern: MVVM-ready (using LiveData and DAOs).

4.2 FRAMEWORK & PATTERN

The application is built on the Android Jetpack suite, primarily utilizing the Room Persistence Library for local data storage and LiveData for reactive UI updates. It follows a modular structure where data logic is separated from the UI layer.

4.3 CORE COMPONENTS

1. Persistence Layer (AppDatabase.java): A Singleton-patterned SQLite database abstraction that manages two main entities: debt and User.
2. Data Access Objects (DAOs):
    - DebtDao.java: Handles complex queries, including a custom @Update query to mark debts as fully paid by syncing the amount_paid with the total amount.
    - UserDao.java: Manages user profile retrieval and updates.
3. Business Logic (DebtCalculator.java): A utility class that iterates through all active debts. It ignores settled items and calculates the net balance based on the DebtType (OWED_TO_ME vs. I_OWE).
4. Data Models:
    - debt.java: The primary entity containing fields for person names, amounts, DebtStatus, PaymentMethod, and timestamps.
    - User.java: Stores user-specific settings and contact information.

4.3.1 Database Schema Room

The application utilizes two primary entities managed by AppDatabase:

1. debt Entity:
    - Fields: debt_id (PK), person_name, amount, debt_type (Enum), status (Enum), date_created, is_settled, and amount_paid.
    - Type Converters: A DateConverter utility is implemented to handle the conversion of Java Date objects into Long timestamps for SQLite storage allowing for seamless storage and sorting of debt creation and due dates.
2. User Entity:
    - Stores profile information such as username, email, and default_currency (defaulting to "KES").

4.3.2 Data Access Objects (DAOs)

- DebtDao: Handles CRUD operations. It includes a specific @Query to mark a debt as paid, updating the is_settled flag, setting the status to 'PAID', and syncing the amount_paid to match the total amount in a single transaction.
- UserDao: Uses LiveData to observe user profile changes, allowing the UI to reactively update the username or preferences.

4.3.3 Core Logic

The DebtCalculator.java utility acts as the financial engine. It iterates through all non-settled debts and performs the following calculation: $$\text{Total Balance} = \sum (\text{Remaining Owed to Me}) - \sum (\text{Remaining I Owe})$$ This ensures the user sees a single, actionable number on their dashboard.

4.4 Status and Enum Definitions

The project utilizes strongly-typed Enums to maintain data integrity:

- DebtType: OWED_TO_ME, I_OWE.
- DebtStatus: PENDING, PAID, PARTIALLY_PAID, OVERDUE.
- PaymentMethod: CASH, MOBILE_MONEY, BANK_TRANSFER.

5 Implementation Details

5.1 Adding a Record

1. The user opens AddDebtActivity via the Floating Action Button.
2. Input validation ensures a name and amount are provided.
3. Upon saving, the app initiates a background thread (using new Thread()) to insert the record into the database, ensuring the UI remains responsive.
4. Utilizies RadioGroup for debt categorization and background threads for database insertion to prevent UI freezing

5.2 Monitoring & Calculation

1. MainActivity observes the DebtDao.getAllDebts() LiveData stream.
2. Whenever a change is detected in the database, the DebtAdapter refreshes the RecyclerView.
3. Simultaneously, the DebtCalculator processes the updated list to refresh the "Total Balance" display at the top of the screen.

5.3 Settling a Debt

Users can interact with records through the DebtAdapter:

- Short Click: Displays an AlertDialog with full details (Due Date, Date Added).
- Long Click: Opens an action menu. Selecting "Mark as Paid" triggers an asynchronous update that sets the status to PAID, toggles is_settled to true, and updates the date_updated timestamp.
- Visual Feedback: Settled debts are rendered with a 0.5f alpha (transparency) to distinguish them from active debts.